

Symetra: Visual Analytics for the Parameter Tuning Process of Symbolic Execution Engines

D. Hong , M. Kim , S. Cha , and J. Jo 

Sungkyunkwan University, Republic of Korea

Abstract

Symbolic execution engines such as KLEE automatically generate test cases to maximize branch coverage, but their numerous parameters make it difficult to understand the parameters' impact, leading the user to rely on suboptimal default configurations. While automated tuners have shown promising results, they provide limited insights into why certain configurations work well, motivating the need for Human-in-the-Loop approaches. In this work, we present a visual analytics system, Symetra, designed to support Human-in-the-Loop parameter tuning of symbolic execution engines. To handle a large number of parameters and their configurations, we provide two complementary overviews of their impact on branch coverage values and patterns. Building on these overviews, our system enables collective analysis, allowing the user to contrast groups of configurations and identify differences that may affect branch coverage. We also report on case studies and a Human-in-the-Loop tuning process, demonstrating that experts not only interpreted parameter impacts and identified complementary configurations, but also improved upon fully automated approaches in both branch coverage and tuning efficiency.

CCS Concepts

• **Human-centered computing** → **Visual analytics**; • **Software and its engineering** → **Software testing and debugging**;

1. Introduction

One important research question in software engineering is how to ensure that a program is robust to arbitrary inputs. This is typically accomplished by generating test cases, executing them, and checking for uncaught exceptions or violated assertions. Symbolic execution [GKS05, CGP*08, SMA05, CDE08] is a powerful method that automatically generates such test cases. The key idea is to systematically explore the program's execution paths by treating input variables as symbolic values and generating inputs that exercise symbolic branches. Various symbolic execution engines, such as KLEE [CDE08], Crest [BS08], or Sage [GLM12], have been researched and adopted.

A critical challenge in effectively leveraging these engines lies in configuring the appropriate parameters that influence their behavior and thus branch coverage. Modern engines incorporate diverse, independent techniques, such as search strategies [CHB*22, HSTV21] and constraint solving [KBC21, SCM*24] that come with numerous tunable parameters. As a result, it has ironically become more difficult to use the engines effectively due to the large number of parameters. For instance, KLEE [CDE08], a widely used symbolic execution engine, offers tens of tunable parameters spanning binary, continuous, and nominal types. This creates a complex, high-dimensional search space with non-linear interactions between parameters, making the tuning process challenging. Furthermore, a single configuration may not be optimal in terms of

branch coverage; the user often requires a set of complementary configurations (i.e., configurations whose test cases cover different sets of branches) to further improve branch coverage. For example, a configuration with the Breadth-First Search (BFS) strategy may provide high overall branch coverage, while one with a Depth-First Search (DFS) strategy may better explore deeply nested branches.

In this work, we aim to enable the user to effectively leverage symbolic execution engines by providing a visual analytics approach to the parameter tuning process. We observed that most users rely on command-line interfaces and general spreadsheet software to manually sift through thousands of configurations. Although automated tuners for the engines [HSTV21, CHB*22, CLLO22] have shown promising results, our domain analysis revealed four critical limitations of these fully automated approaches. First, the user rarely develops an understanding of why certain configurations work well, which would help them use the engines more efficiently in the future, such as when testing newer versions of the same software. Second, even though a small number of configurations can achieve high branch coverage if they target different parts of the program (i.e., complementary configurations), the discovery of such configurations occurs only by chance and is not well understood. Third, although existing tuners are designed to balance exploration and exploitation, the complex search space often leads to early saturation in branch coverage, i.e., repeatedly generating configurations that test similar code regions. Human guid-

ance can help steer the search toward unexplored regions and break such saturation. Finally, certain configurations can result in testing failures, yielding zero branch coverage and wasting computational resources (e.g., running for several minutes without testing any branch). However, existing automated approaches do not provide meaningful insights into their causes. These limitations motivate Human-in-the-Loop approaches where visual analytics bridges automated optimization and human understanding.

To tackle these challenges, we present *Symetra*, a visual analytics system that helps the user analyze and understand the parameter tuning process for symbolic execution engines. We demonstrate that the tuning process can be enhanced not only in a qualitative sense (i.e., providing insights that guide further improvements) but also in a quantitative sense (i.e., achieving higher branch coverage faster). To our knowledge, *Symetra* is the first visual analytics system designed specifically for the tuning process of symbolic execution engines, addressing a critical need in the software testing community where visual analytics has been largely absent. In collaboration with two domain experts, we identified user tasks and requirements that guided our design study. We validated our design through three complementary evaluations: qualitative case studies and expert interviews to assess the system's analytical capabilities, and a quantitative evaluation demonstrating that Human-in-the-Loop parameter tuning improves fully automated approaches.

The contributions of this paper are:

1. Design and development of *Symetra*, the first visual analytics system for Human-in-the-Loop parameter tuning of symbolic execution engines,
2. Abstraction and characterization of the domain situation in symbolic execution-based software testing, and
3. Qualitative and quantitative evaluation of *Symetra* through case studies, expert interviews, and a quantitative demonstration of Human-in-the-Loop parameter tuning.

2. Related Work

In this section, we survey relevant research and systems from both information visualization and software engineering.

2.1. Parameter Tuning for Symbolic Execution

Parameter tuning in symbolic execution engines has been a critical challenge since they became a mainstream testing technique [BEL75,BCD*18,Kin76]. While symbolic execution engines such as KLEE [CDE08] have become widely adopted in academia and industry [CGK*11,CS13], maximizing their performance remains challenging due to the complex parameter space that must be tuned for each target program [LSWL13,TMRC18,WZW*15].

Early parameter tuning approaches were mostly manual and relied heavily on default settings. For instance, many researchers simply adopted KLEE's default parameter values that were originally tailored for GNU Coreutils [CO20,CHZ*18,DWC18,PKR19,WZW*15], even when testing entirely different programs. Others attempted to manually tune specific parameters while keeping the rest at their default values [KBC21,LSWL13,PMZC17,TMRC18,YYG*18], but this approach proved suboptimal given the enormous search space created by the combination of parameters, many of which are non-boolean.

Recent research has made progress toward automated parameter tuning. SymTuner [CLLO22] represents a significant advance by introducing an adaptive learning algorithm that automatically adjusts parameter values during symbolic execution. The system evaluates performance with sampled parameter values and refines probability distributions of the sample spaces based on the results. Similar approaches, such as ParaDySE [CHB*22] and Learch [HSTV21], focus on tuning a specific parameter of the engines (i.e., search strategy) while leaving tens of other important parameters for the user to tune manually. While these automated approaches are effective in improving branch coverage [CDE08,Now19], the lack of visual support makes it difficult for the user to understand the tuning process and derive insights for future improvements. To address this gap, we present *Symetra*, a system that complements fully automated approaches by making the tuning process more transparent.

2.2. Visual Analytics Systems for Parameter Tuning

Visual analytics tools for (hyper)parameter tuning have been extensively studied, particularly in the context of hyperparameter optimization (HPO) for machine learning models. Many of these systems focus on supporting visual analytics to facilitate understanding of parameter configurations tested during the process. For instance, HyperTendril [PNKC21] enables visual exploration of search results from multiple HPO algorithms. A wide range of visualization techniques has been introduced in this context, such as parallel coordinates between parameters representing a configuration as a polyline [GSM*17,Mic21,PNKC21,ASY*19,LLN*18], scatterplots with dimensionality reduction [CMKK21] to provide an overview of configurations across performance metrics and simpler, more familiar visualizations such as bar charts [WMJ*19,LCW*18]. Recent advancements have shifted towards interactive tuning, emphasizing a tighter integration between visual interfaces and search algorithms for human steerability. For example, ATM-Seer [WMJ*19] allows the user to interactively refine the search space during the tuning process, providing greater control.

However, these systems cannot be directly applied to the parameter tuning process in software engineering. The first reason is scalability. Prior systems are typically designed to handle a relatively small number of parameters and configurations, often fewer than ten parameters and hundreds of configurations. This choice may stem from the long time required to evaluate each configuration. In contrast, modern symbolic execution engines have tens of parameters, and each configuration can be tested within a few minutes, leading to thousands of configurations per experiment. Consequently, scalability becomes critical; for instance, using parallel coordinates, as employed in prior work, may not be effective due to the high dimensionality and number of configurations.

The second reason is the unique domain situation of software testing, which we elaborate on in Section 3. Unlike in machine learning, where configurations produce a single performance metric, e.g., accuracy, the configurations in software testing produce a vector with each dimension indicating whether a specific branch was tested, requiring a tailored visualization interface. Furthermore, it is possible to merge test cases generated by different configurations to complement one another to improve branch coverage,

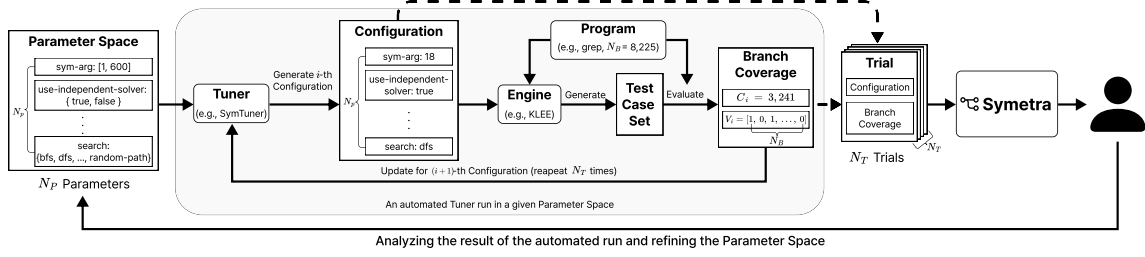


Figure 1: Human-in-the-Loop Parameter Tuning. The gray box represents an experiment where a tuner iteratively optimizes configurations to maximize the branch coverage of the given program. The results of an experiment are trials, pairs of a configuration and its branch coverage, represented as branch coverage vectors (V_i) and branch coverage values (C_i), which are visualized in Symetra. The user then analyzes the results and refines the parameter space for subsequent experiments.

an important user task that is infeasible in machine learning and, therefore, not supported by previous systems.

In parallel, a few studies in software engineering [ABB*19, ZTHD22, HVM15] have proposed visualization systems to aid in understanding and navigating symbolic execution paths and states. For instance, SymNav [ABB*19] introduces a visual analytics approach to assist security analysts in exploring symbolic execution traces, using parallel coordinates and control flow graphs to visualize critical paths in the execution tree. Although these studies offer insights into understanding execution traces and paths, they do not address the challenges associated with parameter tuning for symbolic execution engines. To our knowledge, Symetra is the first visual analytics system specifically designed for parameter tuning of symbolic execution engines, addressing a gap where neither general visualization tools nor existing HPO visualization systems can adequately support the unique requirements.

3. Domain Analysis

We collaborated with two researchers from a university for ten months: E1 with 11 years and E2 with 3 years of experience in symbolic execution. We held monthly meetings where we observed their workflows, discussed their goals and tasks, and iteratively refined them into abstract forms. Each meeting, we presented updated prototypes, collected feedback on both the task abstraction and the visual design, and revised accordingly.

3.1. Background and Terminology

In software engineering, the quality of a test case set is assessed by counting the number of program branches executed by at least one test case in the set, a metric referred to as its **branch coverage**; note that a branch may be exercised multiple times by different test cases in the set, but it is counted only once. As manually crafting a test case set requires a significant amount of time and labor, researchers have developed an automated technique called **symbolic execution**, which analyzes a program, treating the input variables as symbols, and generates the input that makes a certain part of the program executed [Kin76]. KLEE [CDE08] is one of the most widely used symbolic execution engines that has been actively researched in recent years (hereafter, **engine**). From a high-level view, it takes two inputs: **program** and **parameters**. A program is specified as source code, and parameters are configurable options that determine the engine's behavior when generating test

cases. For example, a *search* parameter determines how the engine traverses branches, which could be one of *bfs* or *dfs*.

It is known that choosing the proper parameters plays an important role in improving the branch coverage of the generated test set, which may be different program-by-program [CHB*22, CLLO22]. To automate this parameter tuning process, researchers, including our domain experts, have focused on developing a tuner program (hereafter, **tuner**) that searches for parameters that maximize branch coverage for a given program.

A tuner optimizes the parameters by interacting with the engine in an iterative manner, which is depicted in the gray box of Figure 1. The tuner first randomly samples parameter values from the given parameter space. Then, it runs the engine with those parameter values for a fixed time, generating a test case set. Finally, a branch coverage of the generated set is evaluated, and the tuner updates its internal state to better sample parameter values. This process is repeated for a certain time or iterations. In a sense, the tuning process can be understood analogously to HPO in machine learning; our domain experts use an automated tuner (an HPO algorithm) that maximizes the branch coverage (accuracy) of the generated test case set (trained model) for a specific program (dataset).

We define an **experiment** as a single run of a tuner for the given target program and parameter space with N_P parameters. Figure 1 shows an example of eight parameters. In our study, we used a state-of-the-art algorithm, SymTuner [CLLO22], as a tuner. An experiment runs the engine multiple times, and we call each run a **trial**. We denote a trial by an integer $i \in [1, N_T]$, where N_T represents the total number of trials in the experiment. For each trial, the tuner determines the values of the parameters, which we define as a **configuration**; a configuration is a key-value dictionary of size N_P where keys are parameter names and values are the parameter values, e.g., $\{\text{"sym_arg"} : 100, \text{"search"} : \text{"bfs"}, \dots\}$. After the engine is run on a configuration, the tuner evaluates the branch coverage of the generated test case set. The branch coverage is represented in two ways: as a **branch coverage vector** and a **branch coverage value**. The branch coverage vector of trial i , denoted as $V_i \in \{0, 1\}^{N_B}$, is a binary vector of length N_B where $V_{i,j}$ is a boolean value indicating whether the j -th branch has been executed at least once by the test case set of trial i and N_B is the number of branches in the target program. The branch coverage value of trial i , C_i , is defined as the total number of branches covered (i.e., $C_i = \sum_{j=1}^{N_B} V_{i,j}$).

Among the 148 parameters available in KLEE, we included 61 parameters (i.e., $N_P = 61$) in the parameter space by filtering out those that our domain experts deemed unimportant (e.g., “--help”, “--version”); see the supplementary material for details. The number of trials in an experiment, N_T , typically ranges from a few hundred to a few thousand, with each trial limited to two minutes. We used widely adopted benchmark programs such as gawk ($N_B = 10,720$), gcal ($N_B = 15,799$), and grep ($N_B = 8,225$).

Given the large number of trials generated in an experiment, analyzing them individually is impractical. To address this, we introduce the concept of a **trial group**, a subset of trials selected based on shared characteristics, such as similar branch coverage vectors or common parameter values. This abstraction enables collective analysis at scale. Formally, a trial group $g = \{i_1, \dots, i_k\} \subset [1, N_T]$ is a subset of trials. To quantify how exhaustive its test cases are, we define its **accumulated branch coverage value**, denoted as $accum(g)$. First, we define its branch coverage vector as $V_g = \bigvee_{i \in g} V_i$ where $\bigvee$ denotes the element-wise logical OR operation. Then, $accum(g)$ is defined as the total number of unique branches covered (i.e., $accum(g) = \sum_{j=1}^{N_B} V_{g,j}$). We define two trial groups as **complementary** if the accumulated branch coverage increases when the two groups are merged. Formally, given two trial groups (g_1, g_2), their complementarity $com(g_1, g_2)$ is defined as the coverage gain, $accum(g_1 \cup g_2) - \max(accum(g_1), accum(g_2))$.

Figure 1 depicts the human-in-the-loop parameter tuning process supported by Symetra. While conventional automated tuning operates within a fixed parameter space, Symetra enables users to analyze the outcomes of automated runs and iteratively refine the parameter space to improve efficiency and robustness, for example, by excluding ineffective parameters, removing failure-inducing values, and adjusting default settings.

3.2. User Goals and Tasks

Our target users are researchers developing novel tuning algorithms to improve branch coverage and reduce the time required for test case generation. They have two goals: to understand the parameter configurations produced by the tuner and their impact on branch coverage (**G1**), and to improve branch coverage by identifying complementary parameter values or configurations (**G2**).

During the collaboration, we observed critical inefficiencies in existing workflows for achieving these goals. In the absence of visual support, the user relied on ad hoc practices: manually parsing *gcov* files (log files used to measure branch coverage) into spreadsheet programs, aggregating results in ways that obscured important details, and manually mapping code lines to their IDs. These practices fail to address the four critical limitations outlined in Section 1. We identified five user tasks as follows:

T1. Identifying the parameters and their defaults with the greatest effects on branch coverage. The experts’ primary task was to identify which parameters and their specific values had a significant positive or negative effect on branch coverage. They were particularly interested in ranking parameters by significance to prioritize a smaller subset of parameters in the tuning process for efficiency. They were also interested in comparing the performance of parameter values against the default values, recognizing that the default configuration might be suboptimal for the target program.

T2. Exploring the branch coverage vectors of the tested configurations. Beyond identifying the effect of individual parameters, the experts aimed to analyze how the tested configurations covered branches in a program. Different configurations covering similar sets of branches could indicate redundancy. E1 highlighted this, stating “*Understanding configurations that cover similar program branches can help us avoid wasting computational resources on redundant explorations.*” Conversely, a configuration with a relatively low branch coverage value could still be deemed important if it exercises branches rarely covered by other configurations.

T3. Clustering similar trials. Another task is to cluster similar trials, as they experienced that individually analyzing thousands of trials in an experiment was ineffective. They wanted to cluster the trials into trial groups based on (1) branch coverage value, (2) branch coverage vectors, and (3) the value of a specific parameter.

T4. Comparing trial groups. The experts aimed to understand how trial groups differed from each other in terms of parameter configurations and branch coverage vectors, which could explain the differences in branch coverage. For instance, E2 remarked, “*If we compare two high-performing groups, we want to understand why they are both effective but potentially in different ways.*”

T5. Evaluating the impact of merged trial groups. The experts aimed to investigate whether merging the generated test case sets from different configurations could improve branch coverage. Specifically, they sought to evaluate the improvement in branch coverage achieved by merging test case sets from multiple trials.

4. The Symetra System

4.1. Design Goals

During the design process of Symetra, we established four design goals to support the user tasks effectively as follows:

DG1. Provide an overview of the parameters. Our first goal is to design a comprehensive overview that allows the experts to effectively identify and analyze the influence of parameters on branch coverage. This overview should present the essential information needed to accomplish **T1**, including the magnitude of a parameter’s impact, the effect of each parameter value on branch coverage, and a comparison of their performance relative to the default value.

DG2. Facilitate exploration of branch coverage vectors. Our second design goal is to provide an overview of branch coverage vectors (V_i) to help identify similar or distinct coverage patterns (**T2**), which cannot be revealed by simple branch coverage values (C_i). For instance, two trials might have the same branch coverage value while covering entirely different parts of the program.

DG3. Support collective and comparative analysis of trials. To effectively handle a large number of trials, we aim to provide mechanisms for creating and managing trial groups (**T3**). We also support a comparative analysis between these trial groups (**T4**) and show the increase in the branch coverage value if we merge their test cases (**T5**).

DG4. Enable tuner-agnostic analysis. While we used a certain tuner for this study, our experts also want to analyze the results of other tuners, such as Learch [HSTV21] and ParaDySE [CHB*22].

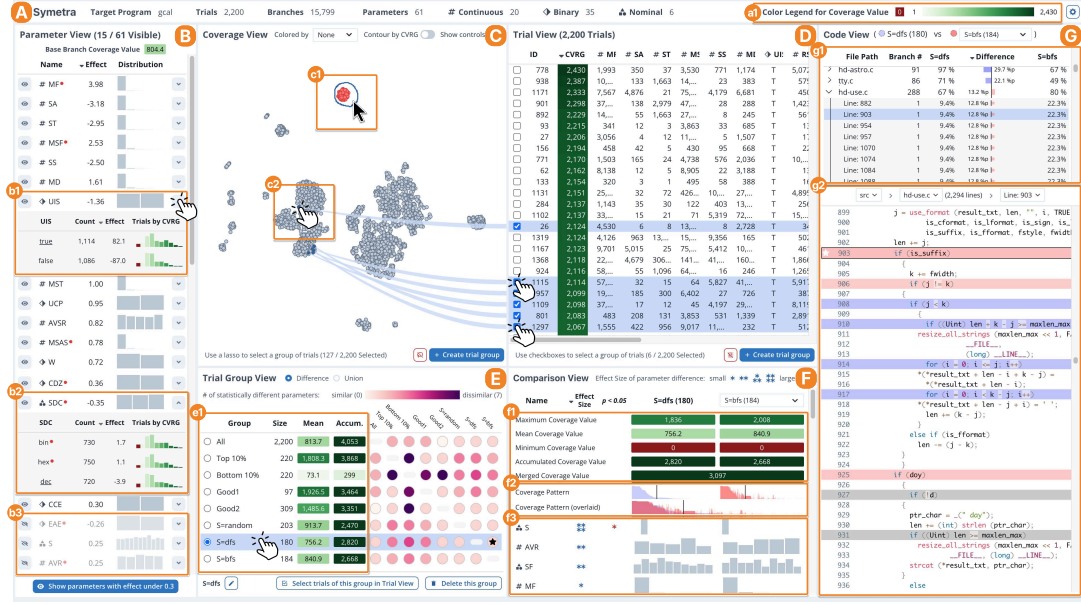


Figure 2: The interface of Symetra. The header (A) shows statistics and the legend for branch coverage values. Two overviews, the Parameter View (B) and the Coverage View (C), provide overviews of parameters (i.e., the input to the engine) and branch coverage (i.e., the output from the engine). The user is making a trial group in the Trial View (D), and the Trial Group View (E) provides a summary of created trial groups, which could be compared through the Comparison View (F) and Code View (G). ($goal$, $N_B = 15,799$, $N_T = 2,200$)

We opted for a tuner-agnostic approach to ensure broader applicability rather than focusing on a specific tuning algorithm. As a result, we treat the tuner as a black box and visualize the relationships between parameter configurations and branch coverage, independent of the underlying tuning algorithm.

4.2. User Interface

The user interface of Symetra consists of six views: Parameter View, Coverage View, Trial View, Trial Group View, Comparison View, and Code View (Figure 2). The system provides two types of overviews; the Parameter View offers an overview of parameters and their effects (i.e., the input to the engine), while the Coverage and Trial Views provide an overview of branch coverage and trials (i.e., the output from the engine). The remaining views support comparative analysis on trial groups. The header (Figure 2-A) displays simple statistics of the experiment, including the number of trials, branches, and parameters of each type. We use a consistent colormap (Figure 2-a1) for branch coverage values across all views, with dark red indicating failed trials.

4.2.1. Parameter View

The Parameter View provides an overview of the effect of the parameters on branch coverage values (Figure 2-B, DG1), supporting T1. To gauge the importance of each parameter, we fit an XGBoost [CG16] model as a surrogate model to approximate the relationship between parameter values and branch coverage. We initially considered more transparent approaches, such as fitting a linear regression model and using its coefficients for parameter importance. Still, we found out that this approach was not capable of capturing the non-linear relationship between parameters, resulting in poor goodness of fit. In our empirical testing, XGBoost consistently

outperformed other models in terms of performance and computational efficiency; see the supplementary material for details.

After fitting the model, we use SHAP (Shapley Additive exPlanations) values [LL17] to quantify the contribution of each parameter to branch coverage. These values represent the average contribution of each parameter value to the overall branch coverage, which can be added to the base coverage value (shown at the top of the Parameter View, Figure 2-B). The effect of a parameter is computed as the weighted mean of the effects of its individual values, where the weight is the number of trials using each value. For example, if the parameter MF has an effect of +3.98 in Figure 2-B, its values, on average, contributed an additional 3.98 branches to the base coverage value of 804.4.

Visual Encoding. The Parameter View lists parameters in a tabular format, with each row displaying the type, name, and effect of a parameter, ordered by magnitude (T1). The rightmost column shows a histogram of parameter value frequencies. The user can click on a row to see the effect of each parameter value (Figure 2-b1). The default value is underlined, and values outperforming the default are marked with a red dot (Figure 2-b2). The user can also control the visibility of parameters through checkboxes, which is linked to the Trial View (Figure 2-b3). By default, parameters with an absolute effect below 0.3 are hidden.

4.2.2. Coverage View

Visual Encoding. The Coverage View provides an overview of branch coverage vectors of trials (Figure 2-C, DG2), supporting T2. We project trials with similar branch coverage vectors into nearby positions using UMAP [MHSJ18] with Jaccard similarity (i.e., $dist(i, j) = 1 - |V_i \cap V_j| / |V_i \cup V_j|$), enabling the user to iden-

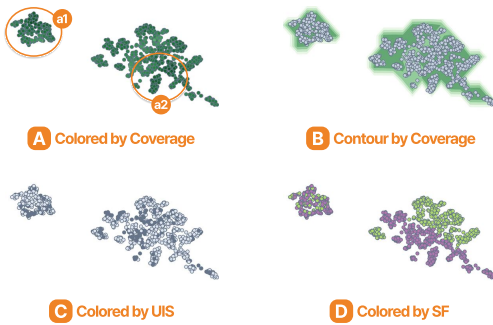


Figure 3: Branch coverage values are shown through point coloring (A) or density map contours (B) using the global colormap (Figure 2-a1). Points can be color-coded by a selected parameter to analyze coverage-parameter correlations, with color schemes adapting to parameter types: UIS (C) and SF (D). ($grep$, $N_B = 8,225$, $N_T = 2,200$)

tify clusters of trials that cover similar branches (Figure 3-a1). We use a conventional scatterplot with each point representing a trial whose coordinates are determined by UMAP. The user can also use different dimensionality reduction techniques or distance measures, configurable in the interface.

The user can color the points by coverage values (Figure 3-A) or use overlaid contours (Figure 3-B). For example, in Figure 3, a1 and a2 both show high coverage values but appear separated, indicating they test different branches. To support correlation analysis between parameter values and coverage vectors, we also allow the user to color-code points based on a selected parameter (Figure 3-C and D). The user can select trials using the lasso tool (Figure 2-c1) and group them by clicking the “Create Trial Group” button (T3) for collective analysis (T4). Clicking on a point scrolls the Trial View to the corresponding row, with explicit visual links for navigation (Figure 2-c2).

Design Alternative. While parallel coordinates plots (PCPs) are commonly used in HPO visualization systems [PNKC21, Mic21, GSM* 17], we opted against PCP for two reasons. First, our experiments involve tens of parameters and thousands of trials, resulting in severe visual clutter. Second, PCPs are designed to reveal similar configurations, whereas our experts focused on complementary configurations (T5) with dissimilar coverage vectors, which are not effectively shown when results are summarized on a single axis.

4.2.3. Trial View

Visual Encoding. The Trial View provides the details of individual trials in a tabular format (Figure 2-D). Each row represents a trial, showing its ID, color-coded branch coverage value (C_i), and configuration. The user can select trials by checking checkboxes and group them using the “Create Trial Group” button (T3). The view is linked to the Coverage View; selected rows are highlighted with connection marks in both views.

4.2.4. Trial Group View

Serving as a starting point of collective and comparative analysis (DG3), the Trial Group View shows trial groups that the user has

created on the Coverage and Trial Views (Figure 2-E). To facilitate onboarding to analysis, we provide three default groups: *All* group consisting of all trials, and the *Top 10%* and *Bottom 10%* groups, containing 10% of the trials with the highest and lowest branch coverage values, respectively.

Visual Encoding. We juxtapose a table and a heatmap where each row represents a trial group. The table shows the name, size, mean, and accumulated branch coverage value (i.e., $accum(g)$) of each group (Figure 2-e1). The heatmap visualizes pairwise relationships between trial groups in two modes: difference mode encodes the number of statistically different parameters (T4), while union mode shows complementarity $com(g_1, g_2)$ (T5).

The user can click on a row to set it as the **active group** (Figure 2-e1) for the Comparison and Code Views, or click on a heatmap cell to compare the corresponding two groups. Hovering over a row highlights its trials in the Coverage View. We provide tooltips in the heatmap and the interactions for managing trial groups, e.g., naming or removing a group, and reselecting the trials of the active group within the Trial View.

Design Alternative. During the design process, we considered using a node-link diagram, where each node represents a trial group, with the thickness of a link indicating group similarity based on parameter values. However, expert feedback revealed several limitations with this approach: it is not space-efficient, and the thickness of links is not perceptually effective. As a result, we opted for a heatmap visualization.

4.2.5. Comparison View

The Comparison View is designed to facilitate the comparison between trial groups, which can explain their difference in branch coverage (Figure 2-F, DG3). We support a pairwise comparison between two trial groups (T4): the active group (selected via the Trial Group View), and another chosen via a dropdown in the Comparison View. The two groups are shown as juxtaposed columns for side-by-side comparison.

Visual Encoding. We support three types of comparative analysis (Figure 2-f1-f3). First, branch coverage statistics (maximum, minimum, mean, accumulated) are shown for each group, along with the merged coverage value $accum(g_1 \cup g_2)$ to indicate if the two groups are complementary (T5). For example, in Figure 2-f1, merging two groups increases coverage from 2,820 and 2,668 to 3,097, indicating their test cases could be merged to improve branch coverage complementarily. Second, coverage pattern bar charts show the covering frequency of each branch, with overlaid charts highlighting differences between groups (Figure 2-f2). Each bar represents a branch, ordered by covering frequency in the active group. Clicking on a bar navigates to the corresponding line in the Code View. Finally, parameter distributions are compared side-by-side as histograms, with effect size and statistical significance indicated by asterisks (Figure 2-f3). We encode the magnitude of difference using blue asterisks and show a red asterisk if the difference is statistically significant. See the supplementary material for details.

4.2.6. Code View

The Code View enables the user to compare the branch coverage of two trial groups at both the file and code levels (Figure 2-G, DG3).

Type	Name	Description	Default	Values / Ranges
Binary	UIS	Use constraint independence	true	{true, false}
	MF	Only fork this many times. Set to -1 to disable	4096	[0, 350,000]
Continuous	ST	Amount of time to dedicate to seeds, before normal search	5	[1, 700,000]
	SA	Replace by a symbolic argument with length N.	-	[1, 600]
Nominal	S	Specify the search heuristic	random-path	{dfs, ..., random-path}
	SF	.ktest file to be used as seed	-	{1, 2, 3, ..., 10}
	SDC	Sets how bitvector constants are written in generated SMT-LIBv2 files	dec	{dec, hex, bin}
	ST	Select the implementation of switch	internal	{internal, simple, llvm}

Table 1: Examples of 8 out of the 61 parameters in KLEE with their names, full names, descriptions, default values, and possible value ranges, categorized by type: Binary (35 parameters) indicating a parameter with one of the boolean values, Continuous (20 parameters) for a parameter in a continuous range, and Nominal (6 parameters) indicating a categorical parameter whose value is one of the given set.

It is designed to help determine whether the two trial groups are complementary in their coverage (**T5**). It consists of the file panel (Figure 2-g1) and the code panel (Figure 2-g2).

Visual Encoding. The file panel organizes source files in a hierarchical structure, allowing users to expand each file to inspect its internal branches and line-level details. We define two coverage metrics: **file coverage**, the ratio of branches in a source file that are tested at least once by a given group, and **line coverage**, the average proportion of trials within a group that cover the branches on each line. Both metrics are visualized as diverging bar charts placed next to each file or line; longer bars indicate stronger coverage bias toward one group, helping users identify files or lines that are predominantly tested by that group (Figure 2-g1).

In the code panel, differences in line coverage are visualized directly on the source code: each line's background color encodes the magnitude and direction of coverage difference between groups (Figure 2-g2). For example, lines 903 and 906 (red) were covered more by $S=bfs$, while lines 908, 910, and 914 were covered more by $S=dfs$. Lines with no difference appear gray. The panels are linked, and clicking on a file or line navigates between them. Branches selected in the Comparison View are highlighted with a black border.

4.3. Implementation

Symetra was implemented with React.js [Wal24] using d3.js [BOH11] and visx [air24] to build visualization components. The source code and a web demo are available at <https://github.com/skku-idclab/Symetra>. Our system requires five input files: 1) a JSON file specifying parameters, including names, types, default values, and descriptions (accessible via tooltips), 2) a CSV file with configurations and branch coverage values for each trial, 3) a text file having the set of branch IDs covered by each trial, 4) a JSON file indicating the line number and file name at which a covered branch is located, and 5) the source code of the target program. This input structure has been verified as universally applicable across different tuners, enabling tuner-agnostic analysis (**DG4**).

5. Evaluation

We evaluated Symetra using both qualitative and quantitative methods: two case studies examining how experts interact with the sys-

tem, interviews with two extra experts to broaden the feedback, and a Human-In-the-Loop optimization study quantifying the system's impact on tuning performance in terms of efficiency and coverage improvements.

5.1. Case Study

We conducted two case studies to evaluate whether and how Symetra can assist experts in achieving **G1** and **G2**. Although we initially considered a comparative evaluation, the absence of dedicated tools in this context limited such an approach. We believe Symetra can therefore serve as a baseline for future research and instead conducted two case studies. We invited two domain experts, P1 and P2, who each had eight years of experience in software engineering and testing, and both held Ph.D. degrees in software engineering. Neither expert was involved in the design process of our system. P1, a researcher at a company's lab, specialized in automated software testing and focused on developing algorithms for automated bug fixing. P2, a researcher at an AI chip company, had experience generating test cases for a compiler. We selected two widely used benchmark programs in software engineering: `gcal` ($N_B = 15,799$) and `grep` ($N_B = 8,225$). SymTuner [CLLO22] was run for each program over three days, resulting in two datasets, each comprising 2,200 parameter configurations.

During the case studies, the experts explored one of the datasets on our system in a 90-minute session. The session began with a 20-minute interview to understand the participants' experience in software testing and to introduce our system. For the next 60 minutes, the participants used the system to freely explore the dataset, analyzing parameter configurations and their impacts (**G1**) and seeking ways to improve branch coverage (**G2**). In the final 10 minutes, we collected subjective feedback through semi-structured interviews to gather insights into their experience using our system. The study protocol was approved by the Institutional Review Board (IRB).

5.1.1. P1: Discovering Complementary Search Strategies

The Parameter View identified influential parameters and sub-optimal defaults (T1). P1 first observed that MF (maximum forks) exhibited the highest positive effect of +3.98, with values exceeding the default, improving coverage by up to +55.3 branches (Figure 4-a1). He explained that coverage typically improved with

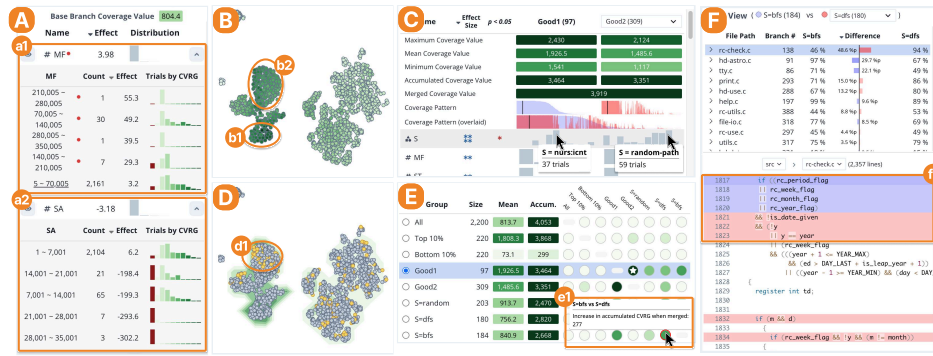


Figure 4: Using the Parameter View (A), P1 examined the effects of the MF and SA parameters. In the Coverage View (B), he identified two groups, Good1 and Good2, that exhibited high overall coverage and similar coverage vectors. The Comparison View (C) revealed that these groups had complementary values for S. By hovering over S=random, he observed that most trials were located in low-coverage regions, though some overlapped with Good2 (D). Based on the trial groups created in the Trial Group View (E), he initiated a comparative analysis in the Code View, focusing on specific parameter values such as dfs, and bfs (F). ($gcal$, $N_B = 15,799$, $N_T = 2,200$)

higher fork limits when sufficient computational resources were available. In contrast, SA (symbolic arguments) showed negative effects when values exceeded 7,000 (Figure 4-a2), which P1 attributed to solver overload and combinatorial explosion. The red dot indicators also revealed that SDC's default value was suboptimal (Figure 2-b2). Reflecting on this, he remarked that having visibility into these parameter effects would significantly assist in defining an effective parameter space.

The Coverage View revealed distinct clusters with different branch compositions (T2). P1 identified two clusters and grouped them as Good1 (97 trials) and Good2 (309 trials) using the lasso tool (T3, Figure 4-b1, b2). Although both achieved high coverage, their separation suggested they covered different branches. In the Trial Group View, P1 was surprised to find only one parameter differed significantly between them (T4). Further investigation in the Comparison View identified S (search heuristic) as the differentiating factor. Good1 used *nurs:icnt* while Good2 used *random-path*. The Comparison View confirmed their complementarity: accumulated coverage of 3,464 and 3,351, respectively, increased to 3,919 when merged (T5, Figure 4-C). Interestingly, P1 had expected trials with *random-path* to be uniformly distributed, as *random-path* explores execution paths stochastically, but found them cohesive in a specific region (Figure 4-d1). He hypothesized that discovering certain critical branches by chance substantially increased coverage, causing these trials to cluster.

The Code View verified strategy-specific patterns at the code level (T5). To further investigate search strategy complementarity, P1 created groups for $S=dfs$ and $S=bfs$ (T3) and found their merged coverage increased from 2,820 and 2,668 individually to 3,097 combined (Figure 4-e1). The Code View revealed the underlying reason: in *rc-check.c*, $S=bfs$ covered branches up to depth 4, while $S=dfs$ penetrated to depth 7 (Figure 4-f1). This confirmed that the two strategies complement each other by targeting different nesting levels—a finding consistent with P1's prior knowledge that could inform parameter space design for other programs.

5.1.2. P2: Diagnosing Failures and Optimizing Seed File Selection

The Parameter View identified seed file values causing failures versus high coverage (T1). P2 first focused on SF (seed file), which exhibited the highest effect of +16.99. Values 3 and 4 positively impacted coverage by up to +174.7 and +157.2 branches (Figure 5-a1), which P2 attributed to special characters and regex patterns enabling comprehensive path exploration. Conversely, values 6, 7, and 8 resulted in frequent failures (Figure 5-a2), likely due to invalid inputs causing premature termination. P2 also examined ST (seed time), finding that values near the default resulted in the lowest effect of -41.6 (Figure 5-a3). He confirmed that values below 10 indicated insufficient seed execution time, while values above 140 would be more appropriate.

The Comparison View enabled systematic failure diagnosis (T4). P2 noted that only 38 out of 2,200 trials failed, suggesting the tuner effectively avoided invalid inputs. To investigate what caused these failures, he created a *Failed* group in the Trial View (T3) and compared it with *Top 10%* in the Trial Group View, finding that 20 parameters differed significantly (Figure 5-b1). The Comparison View revealed that ST (switch type) set to *llvm* was a primary cause of failures. Of 38 failed trials, 29 used *llvm*, while *Top 10%* frequently used *simple* (Figure 5-c1). This systematic comparison allowed P2 to pinpoint specific parameter values to exclude in future experiments.

The Coverage View revealed SF=5 as a potential complementary configuration (T2). When P2 color-coded trials by SF, he discovered that *Top 10%* mainly used values 3 and 4 (Figure 5-d1), while SF=5 trials clustered separately despite not causing failures (Figure 5-d2). He hypothesized these groups could be merged to improve coverage (T5). The Trial Group View confirmed that merging SF=5 with *Top 10%* increased accumulated coverage by 315 branches (Figure 5-b2). The Code View revealed that SF=5 uniquely covered *quotearg.c*, which contained branches requiring quotation mark handling in regular expressions (Figure 5-E). P2 explained that such patterns require specific input structures unlikely to emerge through mutation, concluding that an effective testing

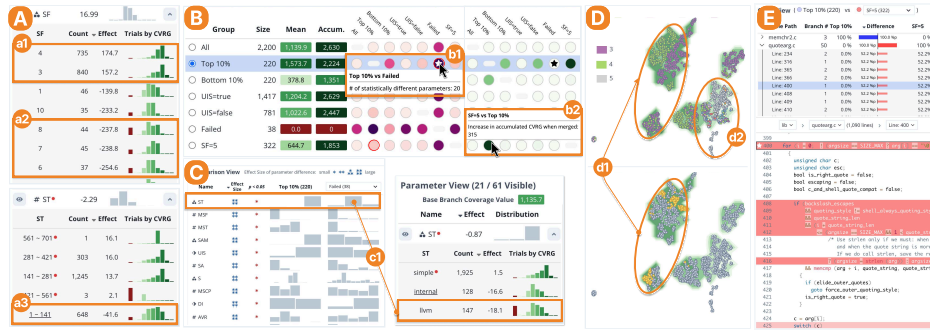


Figure 5: P2 examined the effects of SF and ST in the Parameter View (A). He chose a trial group of interest, Top 10%, in the Trial Group View (B) and compared it with a group of failed trials, Failed, in the Comparison View (C), to investigate parameter values that possibly caused failures. In the Coverage View (D), he color-coded the trials (points) by SF and hovered over Top 10% to highlight only the points in the group, exploring their relationship. The Code View (E) showed the trials in SF=5 mainly covered `quotearg.c`, which were related to handling inputs with quotation marks. ($grep$, $N_B = 8,225$, $N_T = 2,200$)

strategy might involve having different seed files for certain codes, as specific input structures are essential for comprehensive testing.

5.1.3. Results

Across the two case studies, both experts effectively used Symetra's coordinated views to address distinct analytical goals. P1 leveraged the Coverage View to identify complementary search strategies that were not apparent from aggregate branch coverage values alone, whereas P2 used the Parameter View to diagnose failure-causing parameter values. Both case studies demonstrated that the drill-down workflow, which starts from global overviews (Parameter and Coverage Views) and proceeds to detailed comparisons (Comparison and Code Views), supported not only pattern discovery but also verification of their underlying causes at the code level. In particular, the Comparison and Code Views enabled experts to examine differences between trial groups and confirm coverage disparities directly within source files.

Post-study interviews further indicated that both participants regarded the Comparison and Code Views as particularly effective for comparative analysis to improve branch coverage (G2), replacing manual and time-consuming workflows. P1 highlighted the streamlined debugging workflow enabled by the Code View, stating: “Previously, analyzing covered branches required at least three steps: identifying the branch ID, locating the file and line number, and opening the source in a code editor. Symetra reduces this to a single integrated process.”

Both experts also emphasized the usefulness of the Parameter View for understanding how individual parameters and their values influenced branch coverage (G1). The case studies revealed that effective configurations varied substantially across programs. For `gcc`, MF (maximum forks) was the dominant factor influencing coverage, whereas for `grep`, SF (seed file) had the strongest effect. This variation highlights the importance of program-specific analysis when tuning symbolic execution engines.

5.2. Expert Interviews

To broaden feedback beyond the case studies, we interviewed two additional domain experts. P3 was a researcher interested in analyzing differences between human-generated and machine-generated

test cases, while P4 was a researcher focused on developing optimization techniques for symbolic execution engines, particularly search strategies and constraint-solving algorithms. Neither expert was involved in the design process of our system. Each interview lasted one hour: preliminary discussion (20 minutes), a system demonstration (20 minutes), and feedback (20 minutes).

Both experts agreed that the current practices for analyzing engine runs are inefficient; for instance, P3 reported that parsing and interpreting the results of his experiments can take several hours, the main bottleneck in his research. P3 greatly appreciated the Coverage View for highlighting trials with similar coverage values but targeting different code regions (G1). He remarked: “The Coverage View instantly reveals patterns I spent hours searching for manually. Seeing trials cluster by branch coverage vectors can directly support my research on making symbolic execution engines generate more human-like test cases.” He further noted that this overview served as a starting point to drill down into the Code View, where he could examine differences in the tested code regions in more detail. P4, on the other hand, discovered through the Parameter View that common KLEE configurations used in published research were suboptimal; by adjusting the defaults, he was able to increase coverage by more than 100 branches (G2). P4 commented: “Seeing the quantified parameter effects changes how I set up experiments. The default configurations I have been using were not actually optimal for my target programs, requiring careful reconsideration.” Their comments highlight the potential of Symetra to support more efficient workflows in ongoing research.

5.3. Human-in-the-Loop Optimization in Practice

We conducted an experiment to quantify the extent to which Human-in-the-Loop tuning can further improve the branch coverage achieved by a fully automated approach. E1 and E2 analyzed an initial tuner run and refined parameters to improve branch coverage. After each adjustment, we re-ran the experiment to assess improvement over three iterations. We used `gawk` ($N_B = 10,720$) with SymTuner [CLLO22], generating 800 trials per experiment (27 hours each). We measured three metrics: Acc_{800} (the accumulated coverage of all 800 trials), N_{failed} (the number of failed trials with zero coverage), and Acc_{30} (the accumulated coverage of the

first 30 trials, capturing early convergence). The initial experiment, serving as the fully automated baseline, achieved $Acc_{800} = 3,832$, $N_{failed} = 128$, and $Acc_{30} = 3,411$.

Filtering Out Parameters with Little Effects. In the first iteration, while examining the Parameter View, the experts chose to exclude 28 parameters with an average effect below 0.3. They also noticed the parameters with values outperforming the default, highlighted by red dots in the Parameter View, and adjusted their defaults to the values with the greatest positive effect. After the refinement, they achieved $Acc_{800} = 4,093$ (6.8% $\uparrow$), $N_{failed} = 22$ (82.8% $\downarrow$), and $Acc_{30} = 3,651$ (7.0% $\uparrow$).

Removing the Parameter Values Commonly Used in the Failed Trials. In the second iteration, they created a trial group of the 22 failed runs (i.e., $C_i = 0$) through the Trial View. Inspecting the histograms of parameter values in the Comparison View, they identified four parameter values used in the group, which they believed to lead to failures: $KCO=true$, $ST=llvm$, $DI=true$, and $DV=false$. After excluding those values, they achieved $Acc_{800} = 4,379$ (7.0% $\uparrow$), $N_{failed} = 0$ (100% $\downarrow$), and $Acc_{30} = 3,704$ (1.5% $\uparrow$).

Redefining the Defaults. In the third iteration, they found that there were 16 parameters with a better default (i.e., red dots) in the Parameter View, while they changed the defaults in the first iteration. Considering the interaction effect of the changes made in the second iteration, they adjusted the defaults of the 16 parameters to the values that demonstrated the greatest positive effect. After the third refinement, they achieved $Acc_{800} = 4,502$ (2.8% $\uparrow$), $N_{failed} = 0$ (no change), and $Acc_{30} = 3,813$ (2.9% $\uparrow$). Notably, $Acc_{30} = 3,813$ is comparable to $Acc_{800} = 3,832$ from the initial experiment that required 27 hours.

Through three iterations, each guided by analytical evidence from *Symetra*, Human-in-the-Loop tuning achieved coverage of 3,813 within one hour, comparable to 3,832 achieved by the fully automated approach in 27 hours.

6. Discussion and Limitations

Our evaluations demonstrate that *Symetra* enables experts to understand parameter impacts (G1) and identify complementary configurations (G2). The optimization experiment shows that insights from *Symetra* can transfer to future tuning; the refined parameter space achieved the initial 27-hour coverage within one hour, leaving the remaining time to explore additional branches. This improvement suggests that *Symetra* can accelerate testing workflows for similar programs, such as newer versions of the same software.

We learned two lessons from applying visual analytics to a domain where it has been rarely used, software testing. First, engaging with domain experts revealed that users often overtrust default settings that have remained unexamined for over a decade, even though such defaults may not be tailored to their target programs. Visual analytics helped surface these hidden assumptions and motivated experts to adopt more effective parameter choices. As parameter optimization is used across many computational domains, this line of research warrants broader exploration to support more informed and transparent tuning practices in other fields as well.

Second, during the design process, we found that our experts were reluctant to use complex visual encodings, such as parallel coordinates, which have been used in previous interactive HPO systems. Because these experts are already cognitively loaded with demanding tasks, introducing unfamiliar or visually intricate representations can easily become overwhelming rather than helpful. As a result, we prioritized simpler and more familiar representations that minimize interpretation overhead, such as bar charts, histograms, scatterplots, and tables. However, as relying solely on simple visualizations may limit efficiency in the long run, it is important to explore ways to help experts gradually adopt more expressive visual encodings without overwhelming them.

We identify five limitations and directions for future work. First, our system visualizes the results of an experiment without further interaction with the tuner or engine. Integrating them would enable the user to design configurations, launch them, and immediately visualize new trials. Second, we employed XGBoost and SHAP values to quantify parameter impacts. While effective, alternative methods could be explored, such as computing variance to highlight differences across parameter values. Third, displaying six coordinated views simultaneously may increase the initial learning cost. Collapsible panels that progressively reveal views on demand could help new users focus on relevant subsets of the interface. Fourth, our system begins analysis from the parameter space and drills down to code-level details. Supporting the other direction, such as starting from a specific code region to identify relevant parameters, or analyzing how parameter effects vary with experiment duration, remains an open challenge. Lastly, we engaged with six domain experts and three software programs, and the quantitative experiment used a single program, which poses a potential threat to ecological validity. Extending this Human-in-the-Loop approach to other automated tuning systems beyond symbolic execution could further validate its generalizability.

7. Conclusion

We present *Symetra*, a visual analytics system designed to support the parameter tuning process of symbolic execution engines. To address the challenge of managing a large number of configurations, our system supports collective analysis, enabling the user to compare trial groups and examine differences in coverage at both the file and code levels. Our case studies and expert interviews reveal that experts were able to visually analyze the parameter tuning process, gaining insights into parameter impacts and coverage patterns. Finally, our optimization experiment demonstrates that insights from *Symetra* can iteratively improve branch coverage, showcasing the benefits of Human-in-the-Loop parameter tuning.

Acknowledgements

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (RS-2019-II190421, Artificial Intelligence Graduate School Program (Sungkyunkwan University); RS-2024-00438686, Development of software reliability improvement technology through identification of abnormal open sources and automatic application of DevSecOps) and also by the National Research Foundation of Korea (NRF) grant funded by the Korea government (RS-2025-24873100).

References

- [ABB*19] ANGELINI M., BLASILLI G., BORZACCHIELLO L., COPPA E., D'ELIA D. C., DEMETRESCU C., LENTI S., NICCHI S., SANTUCCI G.: Symnav: Visually assisting symbolic execution. In *2019 IEEE Symposium on Visualization for Cyber Security (VizSec)* (2019), pp. 1–11. doi:10.1109/VizSec48167.2019.9161524. 3
- [air24] AIRBNB: visx - a collection of expressive, low-level visualization primitives for react. <https://airbnb.io/visx/>, April 2024. Last Accessed 29 November 2024. 7
- [ASY*19] AKIBA T., SANO S., YANASE T., OHTA T., KOYAMA M.: Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (New York, NY, USA, 2019), KDD '19, Association for Computing Machinery, p. 2623–2631. doi:10.1145/3292500.3330701. 2
- [BCD*18] BALDONI R., COPPA E., D'ELIA D. C., DEMETRESCU C., FINOCCHI I.: A survey of symbolic execution techniques. *ACM Computing Surveys* 51, 3 (May 2018). doi:10.1145/3182657. 2
- [BEL75] BOYER R. S., ELSPAS B., LEVITT K. N.: Select—a formal system for testing and debugging programs by symbolic execution. In *Proceedings of the International Conference on Reliable Software* (New York, NY, USA, 1975), Association for Computing Machinery, p. 234–245. doi:10.1145/800027.808445. 2
- [BOH11] BOSTOCK M., OGIEVETSKY V., HEER J.: D3 data-driven documents. *IEEE Transactions on Visualization and Computer Graphics* 17, 12 (Dec. 2011), 2301–2309. doi:10.1109/TVCG.2011.185. 7
- [BS08] BURNIM J., SEN K.: Heuristics for scalable dynamic test generation. In *Proceedings of the 23rd IEEE/ACM International Conference on Automated Software Engineering* (USA, 2008), ASE '08, IEEE Computer Society, p. 443–446. doi:10.1109/ASE.2008.69. 1
- [CDE08] CADAR C., DUNBAR D., ENGLER D.: Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation* (USA, 2008), OSDI'08, USENIX Association, p. 209–224. 1, 2, 3
- [CG16] CHEN T., GUESTRIN C.: Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (New York, NY, USA, 2016), KDD '16, Association for Computing Machinery, p. 785–794. doi:10.1145/2939672.2939785. 5
- [CGK*11] CADAR C., GODEFROID P., KHURSHID S., PĂSĂREANU C. S., SEN K., TILLMANN N., VISSER W.: Symbolic execution for software testing in practice: preliminary assessment. In *Proceedings of the 33rd International Conference on Software Engineering* (New York, NY, USA, 2011), ICSE '11, Association for Computing Machinery, p. 1066–1071. doi:10.1145/1985793.1985995. 2
- [CGP*08] CADAR C., GANESH V., PAWLOWSKI P. M., DILL D. L., ENGLER D. R.: Exe: Automatically generating inputs of death. *ACM Transactions on Information and System Security* 12, 2 (Dec. 2008). doi:10.1145/1455518.1455522. 1
- [CHB*22] CHA S., HONG S., BAK J., KIM J., LEE J., OH H.: Enhancing dynamic symbolic execution by automatically learning search heuristics. *IEEE Transactions on Software Engineering* 48, 9 (2022), 3640–3663. doi:10.1109/TSE.2021.3101870. 1, 2, 3, 4
- [CHZ*18] CHEN J., HU W., ZHANG L., HAO D., KHURSHID S., ZHANG L.: Learning to Accelerate Symbolic Execution via Code Transformation. In *32nd European Conference on Object-Oriented Programming (ECOOP 2018)* (Dagstuhl, Germany, 2018), Millstein T., (Ed.), vol. 109 of *Leibniz International Proceedings in Informatics (LIPIcs)*, Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, pp. 6:1–6:27. URL: <http://drops.dagstuhl.de/opus/volltexte/2018/9211>, doi:10.4230/LIPIcs.ECOOP.2018.6. 2
- [CLO22] CHA S., LEE M., LEE S., OH H.: Symtuner: maximizing the power of symbolic execution by adaptively tuning external parameters. In *Proceedings of the 44th International Conference on Software Engineering* (New York, NY, USA, 2022), ICSE '22, Association for Computing Machinery, p. 2068–2079. doi:10.1145/3510003.3510185. 1, 2, 3, 7, 9
- [CMK21] CHATZIMPARMPAS A., MARTINS R. M., KUCHER K., KERREN A.: Visevol: Visual analytics to support hyperparameter search through evolutionary optimization. *Computer Graphics Forum* 40, 3 (2021), 201–214. doi:10.1111/cgf.14300. 2
- [CO20] CHA S., OH H.: Making symbolic execution promising by learning aggressive state-pruning strategy. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (New York, NY, USA, 2020), ESEC/FSE 2020, Association for Computing Machinery, p. 147–158. doi:10.1145/3368089.3409755. 2
- [CS13] CADAR C., SEN K.: Symbolic execution for software testing: three decades later. *Communications of the ACM* 56, 2 (Feb. 2013), 82–90. doi:10.1145/2408776.2408795. 2
- [DWC18] DUSTMANN O. S., WEHRLE K., CADAR C.: Parti: a multi-interval theory solver for symbolic execution. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering* (New York, NY, USA, 2018), ASE '18, Association for Computing Machinery, p. 430–440. doi:10.1145/3238147.3238179. 2
- [GKS05] GODEFROID P., KLARLUND N., SEN K.: Dart: directed automated random testing. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation* (New York, NY, USA, 2005), PLDI '05, Association for Computing Machinery, p. 213–223. doi:10.1145/1065010.1065036. 1
- [GLM12] GODEFROID P., LEVIN M. Y., MOLNAR D.: Sage: whitebox fuzzing for security testing. *Communications of the ACM* 55, 3 (Mar. 2012), 40–44. doi:10.1145/2093548.2093564. 1
- [GSM*17] GOLOVIN D., SOLNIK B., MOITRA S., KOCHANSKI G., KARRO J., SCULLEY D.: Google vizier: A service for black-box optimization. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (New York, NY, USA, 2017), KDD '17, Association for Computing Machinery, p. 1487–1495. doi:10.1145/3097983.3098043. 2, 6
- [HSTV21] HE J., SIVANRUPAN G., TSANKOV P., VECHEV M.: Learning to explore paths for symbolic execution. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (New York, NY, USA, 2021), CCS '21, Association for Computing Machinery, p. 2526–2540. doi:10.1145/3460120.3484813. 1, 2, 4
- [HVM15] HONFI D., VOROS A., MICSKEI Z.: Seviz: A tool for visualizing symbolic execution. In *2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST)* (2015), pp. 1–8. doi:10.1109/ICST.2015.7102631. 3
- [KBC21] KAPUS T., BUSSE F., CADAR C.: Pending constraints in symbolic execution for better exploration and seeding. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering* (New York, NY, USA, 2021), ASE '20, Association for Computing Machinery, p. 115–126. doi:10.1145/3324884.3416589. 1, 2
- [Kin76] KING J. C.: Symbolic execution and program testing. *Communications of the ACM* 19, 7 (July 1976), 385–394. doi:10.1145/360248.360252. 2, 3
- [LCW*18] LI T., CONVERTINO G., WANG W., MOST H., ZAJONC T., TSAI Y.-H.: Hypertuner: Visual analytics for hyperparameter tuning by professionals. In *2018 IEEE Workshop on Machine Learning from User Interaction for Visualization and Analytics (MLUI)* (2018), pp. 1–11. doi:10.1109/MLUI52768.2018.10075647. 2
- [LL17] LUNDBERG S. M., LEE S.-I.: A unified approach to interpreting model predictions. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Red Hook, NY, USA, 2017), NIPS'17, Curran Associates Inc., p. 4768–4777. 5

- [LLN*18] LIAW R., LIANG E., NISHIHARA R., MORITZ P., GONZALEZ J. E., STOICA I.: Tune: A research platform for distributed model selection and training. *CoRR abs/1807.05118* (2018). URL: <http://arxiv.org/abs/1807.05118>, arXiv:1807.05118. 2
- [LSWL13] LI Y., SU Z., WANG L., LI X.: Steering symbolic execution to less traveled paths. In *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications* (New York, NY, USA, 2013), OOPSLA '13, Association for Computing Machinery, p. 19–32. doi:10.1145/2509136.2509553. 2
- [MHSG18] MCINNES L., HEALY J., SAUL N., GROSSBERGER L.: Umap: Uniform manifold approximation and projection. *Journal of Open Source Software* 3, 29 (2018), 861. doi:10.21105/joss.00861. 5
- [Mic21] MICROSOFT: Neural Network Intelligence, 1 2021. URL: <https://github.com/microsoft/nni>. 2, 6
- [Now19] NOWACK M.: Fine-grain memory object representation in symbolic execution. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (2019), pp. 912–923. doi:10.1109/ASE.2019.00089. 2
- [PKR19] PANDEY A., KOTCHARLAKOTA P. R. G., ROY S.: Deferred concretization in symbolic execution via fuzzing. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis* (New York, NY, USA, 2019), ISSTA 2019, Association for Computing Machinery, p. 228–238. doi:10.1145/3293882.3330554. 2
- [PMZC17] PERRY D. M., MATTAVELLI A., ZHANG X., CADAR C.: Accelerating array constraints in symbolic execution. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis* (New York, NY, USA, 2017), ISSTA 2017, Association for Computing Machinery, p. 68–78. doi:10.1145/3092703.3092728. 2
- [PNKC21] PARK H., NAM Y., KIM J.-H., CHOO J.: Hypertendril: Visual analytics for user-driven hyperparameter optimization of deep neural networks. *IEEE Transactions on Visualization and Computer Graphics* 27, 2 (2021), 1407–1416. doi:10.1109/TVCG.2020.3030380. 2, 6
- [SCM*24] SHUAI Z., CHEN Z., MA K., LIU K., ZHANG Y., SUN J., WANG J.: Partial solution based constraint solving cache in symbolic execution. *Proceedings of the ACM on Software Engineering* 1, FSE (July 2024). doi:10.1145/3660817. 1
- [SMA05] SEN K., MARINOV D., AGHA G.: Cute: a concolic unit testing engine for c. In *Proceedings of the 10th European Software Engineering Conference Held Jointly with 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering* (New York, NY, USA, 2005), ESEC/FSE-13, Association for Computing Machinery, p. 263–272. doi:10.1145/1081706.1081750. 1
- [TMRC18] TRABISH D., MATTAVELLI A., RINETZKY N., CADAR C.: Chopped symbolic execution. In *Proceedings of the 40th International Conference on Software Engineering* (New York, NY, USA, 2018), ICSE '18, Association for Computing Machinery, p. 350–360. doi:10.1145/3180155.3180251. 2
- [Wal24] WALKE J.: React - the library for web and native user interfaces. <https://react.dev/>, April 2024. Last Accessed 29 November 2024. 7
- [WMJ*19] WANG Q., MING Y., JIN Z., SHEN Q., LIU D., SMITH M. J., VEERAMACHANENI K., QU H.: Atmseer: Increasing transparency and controllability in automated machine learning. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (New York, NY, USA, 2019), CHI '19, Association for Computing Machinery, p. 1–12. doi:10.1145/3290605.3300911. 2
- [WZW*15] WONG E., ZHANG L., WANG S., LIU T., TAN L.: Dase: Document-assisted symbolic execution for improving automated software testing. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering* (2015), vol. 1, pp. 620–631. doi:10.1109/ICSE.2015.78. 2
- [YYG*18] YI Q., YANG Z., GUO S., WANG C., LIU J., ZHAO C.: Eliminating path redundancy via postconditioned symbolic execution. *IEEE Transactions on Software Engineering* 44, 1 (2018), 25–43. doi:10.1109/TSE.2017.2659751. 2
- [ZTHD22] ZIELASKO J., TEMPEL S., HERDT V., DRECHSLER R.: 3d visualization of symbolic execution traces. In *2022 Forum on Specification & Design Languages (FDL)* (2022), pp. 1–8. doi:10.1109/FDL56239.2022.9925664. 3